Session 3: Data Manipulation with pandas

Complied by D K. Muriithi

What is Data Manipulation in Python?

Data manipulation in Python refers to the process of **cleaning, transforming, organizing, filtering, summarizing, and preparing data** so it can be analyzed or used for machine learning and statistical modeling.

In data science and statistics, raw data is often messy. Data manipulation helps convert raw data into a structured and usable format.

Common data manipulation tasks include:

1. **Importing data**
   Loading data from files such as CSV, Excel, databases, or APIs.
2. **Cleaning data**
   Handling missing values, duplicates, incorrect formats, or errors.
3. **Filtering data**
   Selecting rows or columns based on conditions.
4. **Sorting data**
   Arranging data in ascending or descending order.
5. **Transforming data**
   Creating new variables, converting data types, or modifying values.
6. **Aggregating and summarizing data**
   Computing statistics such as mean, median, sum, frequencies, or grouped summaries.
7. **Merging and joining datasets**
   Combining multiple datasets into one.
8. **Reshaping data**

Import necessary libraries

This presentation provides an overview of performing data manipulation in Python using the pandas and numpy libraries. It covers key operations such as filtering, selecting specific columns, modifying variables, sorting, summarizing, chaining operations, and dataset reorganization.

Setting Up the Environment

Before performing data manipulation, ensure that you have the required libraries installed:

In [1]:
```
import pandas as pd                 # Importing pandas for data manipulation
import numpy as np                  # Importing NumPy for numerical computations
import seaborn as sns               # Importing seaborn for data visualization
import matplotlib.pyplot as plt     # Import the pyplot module
import os                           # provides functions for interacting with the op
```

Importing the Dataset

Data is read into a pandas DataFrame using:

In [2]:
```
# Load the GSS (General Social Survey) subset data from a CSV file into a pandas Da
gss = pd.read_csv("GSSsubset.csv")
# Display the first 5 rows of the DataFrame to get a quick overview of the data
gss.head()
```

Out[2]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6

Filtering Rows

The query() function is used to filter data based on conditions:

Filter participants who are Married

Filter the **'gss'** dataframe to include only rows where 'marital' status is **MARRIED**

In [3]:
```
gss_marital = gss.query("marital == 'MARRIED'")
# Display the first 5 rows of the filtered dataframe
gss_marital.head()
#print(gss_marital)
```

Out[3]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6
5	19	MALE	LT HIGH SCHOOL	15703.75	MARRIED	51	68	170	50

Filter rows where age <= 20

Filter the 'gss' dataframe to include only rows where age is less than or equal to 20

In [4]:
```
gss_age = gss.query("age <= 20")
# Display the first 5 rows of the filtered dataframe
```

```
gss_age.head()
```

Out[4]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
67	178	FEMALE	HIGH SCHOOL	2586.50	NEVER MARRIED	20	64	147	31
78	199	FEMALE	HIGH SCHOOL	4803.50	NEVER MARRIED	20	62	160	25
110	290	MALE	HIGH SCHOOL	13856.25	NEVER MARRIED	20	68	230	40
320	836	FEMALE	JUNIOR COLLEGE	20322.50	NEVER MARRIED	19	64	162	40
507	1273	FEMALE	HIGH SCHOOL	3325.50	NEVER MARRIED	20	65	106	36

Selecting Columns

Columns can be selected using loc[]:

Select only the "sex", "income" and "marital" columns from the gss dataframe

In [5]:
```
gss_selected = gss.loc[:, ["sex", "income", "marital"]]
# Display the first 5 rows of the selected dataframe
gss_selected.head()
```

Out[5]:

	sex	income	marital
0	MALE	60967.50	DIVORCED
1	FEMALE	60967.50	MARRIED
2	FEMALE	10161.25	MARRIED
3	FEMALE	17551.25	MARRIED
4	MALE	17551.25	MARRIED

Creating or Modifying Variables

New variables can be created using assign():

In [6]:
```
# import needed libraries
import pandas as pd
import numpy as np

# create a new variable(salary_category)
# This code creates a new column that categorizes income as "High" if over $50,000,
gss_mutated = gss.assign(salary_caterory =np.where(gss["income"] > 50000, "High", "
gss_mutated
```

Out[6]:

	id	sex	degree	income	marital	age	height	weight	hrswrk	salary_
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60	
1	2	FEMALE	BACHELOR	60967.50	MARRIED	26	60	97	40	
2	4	FEMALE	BACHELOR	10161.25	MARRIED	56	68	160	20	
3	14	FEMALE	HIGH SCHOOL	17551.25	MARRIED	40	65	156	37	
4	16	MALE	HIGH SCHOOL	17551.25	MARRIED	56	66	210	6	
...	...	...	...	...	...	...	...	...	...	
989	2531	MALE	HIGH SCHOOL	1478.00	NEVER MARRIED	40	71	230	48	
990	2535	MALE	HIGH SCHOOL	33255.00	DIVORCED	56	72	195	46	
991	2536	MALE	HIGH SCHOOL	8313.75	NEVER MARRIED	24	68	145	40	
992	2537	MALE	HIGH SCHOOL	27712.50	NEVER MARRIED	27	68	180	40	
993	2538	FEMALE	HIGH SCHOOL	15703.75	WIDOWED	71	63	140	48	

994 rows × 10 columns

Sorting Data

The sort_values() function sorts rows based on a column:

Sort the 'gss' DataFrame by the 'income' column in descending order

In [7]:
```
gss_income_arranged = gss.sort_values(by="income", ascending=False)

# Display the sorted DataFrame
gss_income_arranged
```

Out[7]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
159	411	MALE	GRADUATE	158656.952	MARRIED	35	71	164	50
308	806	MALE	BACHELOR	158656.952	MARRIED	47	67	150	60
402	1019	FEMALE	GRADUATE	158656.952	NEVER MARRIED	66	67	150	40
460	1160	MALE	HIGH SCHOOL	158656.952	MARRIED	55	72	185	50
792	2034	MALE	BACHELOR	158656.952	MARRIED	55	66	196	70
...	...	...	...	...	...	...	...	...	...
266	691	FEMALE	BACHELOR	369.500	MARRIED	61	67	170	8
735	1899	FEMALE	HIGH SCHOOL	369.500	DIVORCED	51	67	139	30
379	963	MALE	HIGH SCHOOL	369.500	DIVORCED	59	72	235	70
880	2256	MALE	HIGH SCHOOL	369.500	NEVER MARRIED	44	71	150	40
602	1523	MALE	HIGH SCHOOL	369.500	NEVER MARRIED	26	68	190	30

994 rows × 9 columns

Summarizing Data

Grouping and calculating summary statistics:

1. Group the 'gss' dataframe by the 'sex' column.
2. Calculate the mean of 'income' for each group
3. Reset the index to convert the result back to a regular dataframe

In [8]:
```
gss_summarized = gss.groupby("sex")["income"].mean().reset_index()

# Display the resulting dataframe showing average income by sex
gss_summarized
```

Out[8]:

	sex	income
0	FEMALE	27300.446119
1	MALE	46095.814166

Sorting grouped results:

In [9]:
```
gss_summarized = gss.groupby("degree")["income"].mean().reset_index().sort_values(b
gss_summarized
```

Out[9]:

	degree	income
1	GRADUATE	68457.773552
0	BACHELOR	47738.621187
3	JUNIOR COLLEGE	30580.141304
2	HIGH SCHOOL	28309.892098
4	LT HIGH SCHOOL	18412.616883

Chaining Operations

Multiple operations can be combined using method chaining:

Process the GSS dataset with the following steps:

1. Filter for respondents over 40 years old
2. Select only the columns for degree, income, and sex
3. Create a new column 'income_category' that labels income as 'High' if over $50,000, otherwise 'Low'
4. Sort the results by income in descending order (highest incomes first)

In [10]:
```
gss_processed = (
    gss.query("age > 40")
    .loc[:, ["degree", "income", "sex"]]
    .assign(income_category=lambda x: np.where(x["income"] > 50000, "High", "Low"))
    .sort_values(by="income", ascending=False))
gss_processed
```

Out[10]:

	degree	income	sex	income_category
143	BACHELOR	158656.952	MALE	High
686	GRADUATE	158656.952	MALE	High
86	HIGH SCHOOL	158656.952	MALE	High
92	GRADUATE	158656.952	MALE	High
94	GRADUATE	158656.952	FEMALE	High
...	...	...	...	...
778	HIGH SCHOOL	369.500	FEMALE	Low
735	HIGH SCHOOL	369.500	FEMALE	Low
878	HIGH SCHOOL	369.500	FEMALE	Low
880	HIGH SCHOOL	369.500	MALE	Low
266	BACHELOR	369.500	FEMALE	Low

585 rows × 4 columns

Dataset Reorganization

Accessing specific rows and columns:

In [11]:
```
gss.iloc[9, 1]  # Tenth row, second column
```

Out[11]:
```
'MALE'
```

In [12]:
```
gss.iloc[:, 0]   # First column
```

Out[12]:
```
0         1
1         2
2         4
3        14
4        16
       ...
989    2531
990    2535
991    2536
992    2537
993    2538
Name: id, Length: 994, dtype: int64
```

In [13]:
```
gss.iloc[0, :]  # Tenth row
```

Out[13]:
```
id               1
sex           MALE
degree     BACHELOR
income      60967.5
marital    DIVORCED
age              53
height           72
weight          190
hrswrk           60
Name: 0, dtype: object
```

Subsetting based on conditions:

In [14]:
```
# Filter the gss dataframe to include only rows where income is greater than $50,00
gss[gss["income"] > 50000]
```

Out[14]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	BACHELOR	60967.500	DIVORCED	53	72	190	60
1	2	FEMALE	BACHELOR	60967.500	MARRIED	26	60	97	40
8	28	MALE	BACHELOR	73900.000	MARRIED	57	71	225	40
12	40	MALE	BACHELOR	73900.000	MARRIED	35	69	200	50
14	45	FEMALE	BACHELOR	88680.000	SEPARATED	50	67	188	60
...	...	...	...	...	...	...	...	...	...
932	2383	MALE	HIGH SCHOOL	60967.500	MARRIED	49	68	178	45
951	2419	MALE	LT HIGH SCHOOL	60967.500	MARRIED	48	70	205	65
952	2423	MALE	BACHELOR	60967.500	MARRIED	30	67	195	48
953	2428	MALE	GRADUATE	158656.952	MARRIED	67	70	183	75
967	2474	FEMALE	BACHELOR	158656.952	DIVORCED	38	65	145	45

174 rows × 9 columns

Filter the gss dataframe to only include rows where the "degree" column equals "GRADUATE"

This creates a new dataframe containing only records of people with graduate degrees

In [15]:
```
gss[gss["degree"] == "GRADUATE"]
```

Out[15]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
18	49	FEMALE	GRADUATE	88680.000	MARRIED	32	66	160	40
20	51	FEMALE	GRADUATE	33255.000	MARRIED	52	63	143	40
25	61	MALE	GRADUATE	15703.750	NEVER MARRIED	51	70	190	14
26	64	FEMALE	GRADUATE	60967.500	NEVER MARRIED	47	64	123	40
27	68	MALE	GRADUATE	158656.952	DIVORCED	57	71	180	70
...	...	...	...	...	...	...	...	...	...
927	2374	MALE	GRADUATE	158656.952	DIVORCED	40	70	290	89
940	2400	MALE	GRADUATE	33255.000	MARRIED	38	69	170	50
942	2403	FEMALE	GRADUATE	33255.000	MARRIED	54	62	130	46
953	2428	MALE	GRADUATE	158656.952	MARRIED	67	70	183	75
979	2517	MALE	GRADUATE	40645.000	MARRIED	65	68	165	40

125 rows × 9 columns

In [16]:
```
# Filter the 'gss' dataframe to only include rows where the 'marital' column equals
gss[gss["marital"] == "DIVORCED"]
```

Out[16]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	BACHELOR	60967.50	DIVORCED	53	72	190	60
6	21	FEMALE	HIGH SCHOOL	17551.25	DIVORCED	30	62	115	38
9	30	MALE	BACHELOR	33255.00	DIVORCED	54	71	165	40
10	32	FEMALE	BACHELOR	33255.00	DIVORCED	61	64	128	40
11	38	MALE	HIGH SCHOOL	33255.00	DIVORCED	31	67	150	39
...	...	...	...	...	...	...	...	...	...
969	2480	FEMALE	HIGH SCHOOL	20322.50	DIVORCED	55	65	208	40
970	2482	FEMALE	HIGH SCHOOL	17551.25	DIVORCED	53	70	170	20
974	2500	MALE	HIGH SCHOOL	20322.50	DIVORCED	53	72	200	60
983	2522	FEMALE	HIGH SCHOOL	24017.50	DIVORCED	49	67	200	50
990	2535	MALE	HIGH SCHOOL	33255.00	DIVORCED	56	72	195	46

172 rows × 9 columns

Summary Report

Aggregating data for better insights:

Group the 'gss' DataFrame by the 'sex' column and calculate multiple statistics for the 'income' column: mean, minimum, maximum, median, and standard deviation

In [17]:
```
gss.groupby("sex")["income"].agg(["mean", "min", "max", "median", "std"])
```

Out[17]:

sex	mean	min	max	median	std
FEMALE	27300.446119	369.5	158656.952	20322.5	23871.727298
MALE	46095.814166	369.5	158656.952	33255.0	40527.991908

Group the 'gss' DataFrame by education level ('degree') and calculate summary statistics for the 'income' column, including mean, minimum, maximum, median, and standard deviation

In [18]:
```
gss.groupby("degree")["income"].agg(["mean", "min", "max", "median", "std"])
```

Out[18]:

degree	mean	min	max	median	std
BACHELOR	47738.621187	369.5	158656.952	40645.00	38036.482608
GRADUATE	68457.773552	369.5	158656.952	49882.50	47526.992948
HIGH SCHOOL	28309.892098	369.5	158656.952	24017.50	26671.333894
JUNIOR COLLEGE	30580.141304	1478.0	103460.000	27712.50	20292.993851
LT HIGH SCHOOL	18412.616883	1478.0	73900.000	15703.75	13324.116851

Group the data by 'sex' and 'degree' columns, then calculate the mean income for each group

This creates a Series with a hierarchical index (MultiIndex) where the first level is 'sex' and the second level is 'degree'

In [19]:
```
gss.groupby(["sex", "degree"])["income"].mean()
```

Out[19]:
```
sex     degree
FEMALE  BACHELOR          34269.353010
        GRADUATE          52381.502655
        HIGH SCHOOL       21630.135693
        JUNIOR COLLEGE    22180.077273
        LT HIGH SCHOOL    11350.578125
MALE    BACHELOR          60334.881241
        GRADUATE          82374.545373
        HIGH SCHOOL       34749.177272
        JUNIOR COLLEGE    43066.722973
        LT HIGH SCHOOL    23434.511111
Name: income, dtype: float64
```

Hand-On Exercise: Manipulating the 'mtcars' Dataset

A sample dataset (mtcars) is used for practice:

In [20]:
```
mtcars = pd.read_csv("mtcars.csv")

# Display the first 5 rows of the DataFrame to preview the data
mtcars.head()
```

Out[20]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horse
0	Acura	MDX	SUV	Asia	All	36945.0	33337.0	3.5	6.0	
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	23820.0	21761.0	2.0	4.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	26990.0	24647.0	2.4	4.0	
3	Acura	TL 4dr	Sedan	Asia	Front	33195.0	30299.0	3.2	6.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	43755.0	39014.0	3.5	6.0	

Question 1: Filter the mtcars dataframe to include only rows where the Weight is greater than 3.5

Question 2: Select specific columns from the mtcars dataframe Using .loc to access rows (all rows with :) and specific columns by name "MPG_City", "Cylinders", "Horsepower"

Question 3: Create or modify variables using assign() to add a new column 'Horsepower_cateories' to the dataframe. Apply conditional logic with np.where() to categorize horsepower values, If horsepower > 150, assign "High", otherwise assign "Low"

Question 4: Sort the mtcars DataFrame by the 'Horsepower' column in descending order

Question 5: Group the mtcars dataframe by the "Cylinders" column for each cylinder group, calculate the mean, minimum, and maximum values of the "MPG_Highway" column

In [21]:
```
# Filter the mtcars dataframe to include only rows where the Weight is greater than
mtcars_filtered = mtcars[mtcars["Weight"] > 3.5]

# Display the first 5 rows of the filtered dataframe
mtcars_filtered.head()
```

Out[21]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horse
0	Acura	MDX	SUV	Asia	All	36945.0	33337.0	3.5	6.0	
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	23820.0	21761.0	2.0	4.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	26990.0	24647.0	2.4	4.0	
3	Acura	TL 4dr	Sedan	Asia	Front	33195.0	30299.0	3.2	6.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	43755.0	39014.0	3.5	6.0	

In [22]:
```
# Select specific columns from the mtcars dataframe
# Using .loc to access rows (all rows with :) and specific columns by name
mtcars_selected = mtcars.loc[:, ["MPG_City", "Cylinders", "Horsepower"]]
# Display the first 5 rows of the resulting dataframe
mtcars_selected.head()
```

Out[22]:

	MPG_City	Cylinders	Horsepower
0	17	6.0	265
1	24	4.0	200
2	22	4.0	200
3	20	6.0	270
4	18	6.0	225

In [23]:
```
# Create or modify variables using assign()
mtcars_mutated = mtcars.assign(Horsepower_cateories=np.where(mtcars["Horsepower"] >
mtcars_mutated.head()
```

Out[23]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horse
0	Acura	MDX	SUV	Asia	All	36945.0	33337.0	3.5	6.0	
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	23820.0	21761.0	2.0	4.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	26990.0	24647.0	2.4	4.0	
3	Acura	TL 4dr	Sedan	Asia	Front	33195.0	30299.0	3.2	6.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	43755.0	39014.0	3.5	6.0	

In [24]:
```
# Sort rows using sort_values()
mtcars_sorted = mtcars.sort_values(by="Horsepower", ascending=False)
mtcars_sorted.head()
```

Out[24]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cyl
114	Dodge	Viper SRT-10 convertible 2dr	Sports	USA	Rear	81795.0	74451.0	8.3	
262	Mercedes-Benz	CL600 2dr	Sedan	Europe	Rear	128420.0	119600.0	5.5	
271	Mercedes-Benz	SL600 convertible 2dr	Sports	Europe	Rear	126670.0	117854.0	5.5	
270	Mercedes-Benz	SL55 AMG 2dr	Sports	Europe	Rear	121770.0	113388.0	5.5	
334	Porsche	911 GT2 2dr	Sports	Europe	Rear	192465.0	173560.0	3.6	

In [25]:
```
# Compute summary statistics using groupby() and agg()
mtcars_summary = mtcars.groupby("Cylinders")["MPG_Highway"].agg(["mean", "min", "ma
mtcars_summary
```

Out[25]:

Cylinders	mean	min	max
3.0	66.000000	66	66
4.0	31.889706	23	51
5.0	26.857143	25	28
6.0	25.552632	17	32
8.0	21.885057	12	28
10.0	16.500000	13	20
12.0	19.000000	19	19

Homework Assignment

Exercise A: Import and Explore

- Download a public dataset (german_credit__data)
- Load in python
- Print first few rows and check data types

Exercise B: Filter and Select

- Use filter to extract relevant rows (e.g., "Credit amount>5000")
- Use select to keep only necessary columns

Exercise C: Mutate

Add at least two new computed columns:

- One conditional flag (e.g., male/female sex)

Exercise D: Group and Summarize

- Average value per category
- Count per category
- Sort results by average value descending

Exercise E: Write Your Own Pipeline

- Combine and at least 3 verbs to produce a clean result
- Comment each step clearly

https://cdam.chuka.ac.ke

Thank you